STŘEDNÍ PRŮMYSLOVÁ ŠKOLA

MLADÁ BOLESLAV

# ROČNÍKOVÁ PRÁCE

Michal Prawdzin

Mladá Boleslav 2026

STŘEDNÍ PRŮMYSLOVÁ ŠKOLA

MLADÁ BOLESLAV

# SAMURAI

**Autor: Michal Prawdzin**

**Studijní obor: 18-20-M/01 Informační technologie**

**Vedoucí práce: Jan Urban**

Mladá Boleslav 2026

# Obsah

Obsah ..... 3

1 Úvod ..... 7

2 Obsah práce ..... 9

2.1 Použité technologie ..... 9

2.1.1 Python ..... 9

2.1.2 Pygame ..... 9

2.1.3 Pyinstaller ..... 9

2.1.4 Visual Studio Code ..... 10

2.1.5 Piskel ..... 10

2.1.6 Pixilart ..... 10

2.1.7 Bfxr ..... 11

2.1.8 GitHub Copilot ..... 11

2.2 Herní mapa ..... 12

2.2.1 Vývoj ..... 12

2.2.2 Problém a řešení ..... 12

2.3 Samurai ..... 13

2.3.1 Hitbox ..... 13

2.3.2 Pohyb ..... 13

2.3.3 Grafika ..... 15

2.3.4 Animace ..... 16

2.4 Menu ..... 17

2.4.1 Font ..... 18

2.5 Zvuky ..... 18

2.6 Problémy s vývojem ..... 19

2.6.1 Problém s mechanikou wallgrab ..... 19

2.6.2 Debug ..... 19

2.6.3 Bugy ..... 19

3 Možná budoucí rozšíření ..... 21

3.1 Příběh, konec a prodloužení mapy ..... 21

3.2 Přidání nepřátel ..... 21

3.3 Přidání zkosených platforem a jiných elementů ..... 21

4 Závěr ..... 22

5 Přílohy ..... 23

5.1 Zdroje ..... 23

5.1.1 Font ..... 23

5.1.2 Sound efekty ..... 23

**Prohlášení**

Prohlašuji, že jsem svou ročníkovou práci vypracoval samostatně a použil jsem pouze podklady (literaturu, projekty, SW atd.) uvedené v přiloženém seznamu.

Nemám závažný důvod proti zpřístupňování této ročníkové práce v souladu se zákonem č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon) v platném znění.

V Mladé Boleslavi dne ..... podpis: .....

# 1 Úvod

Cílem této ročníkové práce bylo vytvoření vlastní 2D plošinové hry inspirované známou hrou Jump King[1]. Hlavní myšlenkou projektu bylo navrhnout hru, která bude založená na vypočítaném ovládání postavy, fyzice pohybu a postupném překonávání překážek směrem nahoru. Zároveň bylo cílem přidat do hry i vlastní prvky a mechaniky – například sliding a skákání od stěn.

Hra byla vytvořena v programovacím jazyce Python[2] s využitím knihovny Pygame[3]. Python byl zvolen především kvůli své jednoduchosti. Pygame umožňuje relativně snadno pracovat s grafikou, zvuky a vstupy z klávesnice, což z něj činí skvělý nástroj pro tvorbu menších herních projektů. Vývoj probíhal ve vývojovém prostředí Visual Studio Code[4], které poskytuje přehledné rozhraní a podporu pro práci s Pythonem.

Během vývoje této práce bylo možné naučit se základní principy vývoje her. Mezi tyto principy patří například tvorba herní smyčky, zpracování vstupu od hráče a vykreslování grafiky. Dále bylo nutné řešit fyziku pohybu postavy, detekci kolizí mezi hráčem a platformami a práci s kamerou.

Grafika této hry byla vytvořena pomocí dvou nástrojů – Piskel[5] a Pixilart[6]. Piskel byl využit především pro úpravu velikosti obrázků a práci s plátnem, zatímco tvorba grafiky

---

[1] *NEXILE. Jump King. [online]. 2019. Dostupné z: https://store.steampowered.com/app/1061090/Jump_King/. [cit. 2026-04-28].*

[2] *PYTHON SOFTWARE FOUNDATION. Python. [online]. 2024. Dostupné z: https://cs.wikipedia.org/wiki/Python. [cit. 2026-04-28].*

[3] *PYGAME COMMUNITY. Pygame. [online]. 2024. Dostupné z: https://en.wikipedia.org/wiki/Pygame. [cit. 2026-04-28].*

[4] *MICROSOFT. Visual Studio Code. [online]. 2026. Dostupné z: https://code.visualstudio.com/. [cit. 2026-04-28].*

[5] *Piskel. [online]. 2018. Dostupné z: https://github.com/piskelapp/piskel. [cit. 2026-04-28].*

[6] *Pixilart. [online]. 2026. Dostupné z: https://www.pixilart.com/about. [cit. 2026-04-28].*

probíhala v prostředí Pixilart, které poskytuje přehlednější uživatelské rozhraní pro kreslení.

Zvukové efekty byly vytvořeny pomocí webové aplikace Bfxr[7], která je určena pro generování jednoduchých retro zvuků vhodných pro 2D herní projekty.

[7] *Bfxr. [online]. 2020. Dostupné z: https://www.bfxr.net/. [cit. 2026-04-28].*

# 2 Obsah práce

## 2.1 Použité technologie

### 2.1.1 Python

Programovací jazyk Python je moderní vysokoúrovňový jazyk, který je známý svou jednoduchostí, čitelností a širokým využitím. Používá se v mnoha oblastech informatiky, například při tvorbě webových aplikací, analýze dat, automatizaci nebo vývoji her. Jednou z jeho hlavních výhod je přehledná syntaxe, která umožňuje rychlé pochopení i pro méně zkušené programátory. Díky tomu je Python často využíván ve vzdělávání a při výuce programování.

### 2.1.2 Pygame

Pro vývoj samotné hry byla využita knihovna Pygame. Pygame je open-source knihovna určená pro tvorbu 2D her v jazyce Python. Poskytuje nástroje pro práci s grafikou, zvukem, vstupem z klávesnice a myši a také umožňuje implementaci herní smyčky. Díky tomu je možné relativně snadno vytvořit plně funkční herní aplikaci bez nutnosti používat složité herní enginy.

### 2.1.3 Pyinstaller

Pro vytvoření spustitelné verze hry byl využit nástroj PyInstaller. Jedná se o program, který umožňuje převést Python aplikaci do samostatného spustitelného souboru (.exe), který lze spustit i na počítačích, kde není nainstalovaný Python. Tento nástroj shromažďuje všechny potřebné soubory, knihovny a závislosti do jednoho balíčku.[8]

[8] *PYINSTALLER DEVELOPMENT TEAM. Pyinstaller. [online]. 2026. Dostupné z: https://pyinstaller.org/en/stable/. [cit. 2026-04-28].*

### 2.1.4 Visual Studio Code

Pro vývoj této ročníkové práce bylo použito vývojové prostředí Visual Studio Code. Jedná se o moderní editor zdrojového kódu od společnosti Microsoft, který je dostupný zdarma a podporuje mnoho programovacích jazyků včetně Pythonu. Tento editor je oblíbený díky své jednoduchosti, rychlosti a přehlednému uživatelskému rozhraní.

Visual Studio Code nabízí užitečné funkce, jako je zvýrazňování syntaxe, automatické doplňování kódu nebo integrovaný terminál. Tyto nástroje usnadňují psaní kódu a pomáhají rychleji odhalit chyby. Při práci na projektu bylo využito také rozšíření pro Python, které umožňuje snadné spouštění programu přímo v editoru.

### 2.1.5 Piskel

Piskel je webová aplikace určená pro tvorbu a úpravu pixel art grafiky. Tento nástroj umožňuje pracovat s obrázky na úrovni jednotlivých pixelů, což je vhodné zejména pro vývoj 2D her s jednoduchým nebo retro vizuálním stylem. Piskel nabízí funkce jako kreslení, práce s barvami, vrstvy nebo základní animace.

### 2.1.6 Pixilart

Pixilart je webová aplikace určená pro tvorbu pixel art grafiky. Tento nástroj umožňuje kreslení obrázků pomocí jednoduchého rozhraní, které je přehledné a vhodné i pro začátečníky. Pixilart nabízí základní funkce, jako je práce s barvami, štětci, vrstvami nebo jednoduché animace, což z něj činí vhodný nástroj pro tvorbu grafiky do 2D her.

V této práci byl Pixilart využit především pro samotnou tvorbu grafických prvků hry, například postavy nebo vizuální podoby platforem. Oproti jiným nástrojům nabízí přehlednější uživatelské rozhraní, které usnadňuje kreslení a práci s detaily.

### 2.1.7 Bfxr

Bfxr je jednoduchá webová aplikace určená pro generování zvukových efektů, které se často používají ve hrách, zejména v retro nebo 2D projektech. Tento nástroj umožňuje vytvářet různé typy zvuků, například skoky, výbuchy nebo interakce, pomocí úpravy několika základních parametrů, jako je frekvence, délka nebo hlasitost.

### 2.1.8 GitHub Copilot

GitHub Copilot je nástroj využívající umělou inteligenci, který pomáhá při psaní a úpravě kódu. Funguje tak, že na základě aktuálně psaného kódu navrhuje možné pokračování nebo opravy. Díky tomu může urychlit vývoj a usnadnit řešení problémů.[9]

V této práci byl GitHub Copilot využit především pro debug části kódu. Pomáhal s hledáním chyb a přímo k nakódování některých debug funkcí, Díky tomu byl vývoj efektivnější a bylo možné rychleji řešit problémy, které se během tvorby hry objevily.

[9] *GitHub Copilot. [online]. 2026. Dostupné z: https://docs.github.com/en/copilot/get-started/what-is-github-copilot. [cit. 2026-04-28].*

## 2.2 Herní mapa

### 2.2.1 Vývoj

Tvorba herní mapy se skládala z více kroků a kombinovala jak programování, tak grafickou úpravu. Nejprve byly jednotlivé platformy vytvořeny přímo v kódu. Platformy jsou uložené v seznamu, kde má každá z nich definovanou svou pozici a velikost, což umožňuje správný pohyb postavy mezi nimi.

Před samotnou úpravou platforem bylo nejdříve vytvořeno pozadí mapy. To vznikalo ve webové aplikaci Pixilart po jednotlivých částech, protože aplikace má omezenou velikost plátna. Díky tomu bylo možné vytvořit větší mapu a zároveň si zachovat kontrolu nad detaily.

Po vytvoření základního rozložení platforem byl tento návrh exportován jako obrázek ve formátu PNG. Export byl proveden pomocí kódu, který byl vytvořen s pomocí umělé inteligence. Tento obrázek byl následně importován do aplikace Piskel, kde byl rozdělen na menší části a upraven na požadovanou velikost. Poté byl znovu exportován a importován zpět do Pixilart.

V Pixilartu byly platformy graficky upraveny tak, aby vizuálně odpovídaly stylu a tématu pozadí. Díky tomuto postupu bylo možné zachovat přesné rozložení mapy z kódu a zároveň vytvořit jednotný vizuální vzhled celé hry.

### 2.2.2 Problém a řešení

Tento způsob ale nebyl velmi efektivní. V případě změny souřadnic nebo velikosti platformy v kódu se tato změna musela upravit i v obrázku platforem toto bylo problémem nejméně dvakrát.

Mapu bylo možno vytvořit i využitím tiles. Tímto způsobem by se mapa skládála z čtverečků seřazených podle potřeby. Tento způsob by úplně odstranil problém, který byl překážkou při vývoji této hry. I když je tile systém ve hrách běžně využíván, protože umožňuje snadnější a efektivnější úpravy mapy, v rámci této práce tento způsob nebyl použit, protože je komplikovanější pro implementaci s pygame

## 2.3 Samurai

### 2.3.1 Hitbox

Pro zjednodušení kolizí je postava reprezentována pouze hitboxem, který má tvar obdélníku. Tento hitbox slouží jako jednoduchý model pro detekci kolizí s platformami a stěnami pomocí metody colliderect z knihovny Pygame. Díky tomu je výpočet kolizí rychlejší a zároveň přehlednější.

Pozice postavy je určena souřadnicemi tohoto hitboxu, přičemž samotné sprity jsou vykreslovány na jeho pozici na obrazovce. To znamená, že grafika postavy je oddělena od její fyzikální reprezentace, což usnadňuje práci s pohybem a kolizemi.

Ve hře je také implementována mechanika tvrdého dopadu. Pokud postava při pádu dosáhne určité rychlosti a narazí na horní část platformy, aktivuje se stav omráčení (stun). Tento stav na krátkou dobu znemožňuje hráči pohyb, což zvyšuje obtížnost hry a nutí hráče lépe kontrolovat své skoky.

### 2.3.2 Pohyb

Hra je ovládaná podle základních a hráčům nejvíce známým klávesám. Pro pohyb do stran jsou využity klávesy A a D, mezerník slouží pro skok a C pro slide. Vstup od hráče je v

programu řešen pomocí funkce pygame.key.get_pressed(), která v každém snímku herní smyčky kontroluje stav kláves.

#### 2.3.2.1 Horizontální pohyb

Horizontální pohyb funguje pomocí změny proměnné player_vel_x. Pokud hráč drží buď klávesu A nebo D, zvyšuje nebo snižuje hodnotu player_vel_x. Tato hodnota je omezena, aby nedocházelo k nekonečnému zrychlování. Pokud hráč pustí klávesu, začne se snižovat rychlost postavy pomocí friction, dokud se postava úplně nezastaví. Díky tomu pohyb působí plynule a přidává lehce na obtížnosti. Hodnota friction nemusí být stejná, v této hře je vyrobena ledová část mapy pomocí souřadnic a to přesně tak, že pokud se hráč nachází v rozmezí daných souřadnic, hodnota proměnné friction se může snížit nebo zvýšit. Díky tomu se postava může buď klouzat, nebo zastavit skoro ihned,

Další mechanika horizontálního pohybu je sliding, která se aktivuje, když hráč zmáčkne klávesu C při pohybu do stran. Při slidu se sníží hitbox a zvýší se rychlost postavy. Pomocí slidu se může postava na příklad vyhnout nějakým překážkám nebo překonat delší úsek platformy.

#### 2.3.2.2 Vertikální pohyb

Skákání funguje na základě „nabití“ skoku. Když hráč drží mezerník, postupně se zvyšuj hodnota proměnné jump_power. Zároveň se snižuje hodnota side_jump. To znamená, že čím déle hráč drží klávesu, tím více postava vyskočí nahoru, ale méně do strany.

Naopak, když hráč podrží mezerník jen chvíli postava vyskočí méně do výšky, ale více do strany. Postava může skočit dvěma způsoby – buď jump_power dosáhne maximální hodnoty jump_max a postava skočí sama, nebo hráč pustí mezerník.

Další mechanikou je wall jump. Pokud hráč podrží mezerník před kolizí s platformou, aktivuje se stav wall_grab. V tomto stavu se postava přichytí ke stěně a pomalu začne klouzat dolů.

Pokud hráč v tomto stavu pustí mezerník, provede se wall jump. Při wall jumpu se nastaví hodnota vertikální rychlosti směrem nahoru a horizontální rychlost opačným směrem od stěny. Díky této mechanice může struktura mapy být obtížnější a rozmanitější.

Je tu zde také mechanika tvrdého dopadu. Pokud postava přesáhne danou rychlost pádu a dotkne se horní části jakékoliv platformy, postava se omráčí (stun), tato mechanika dočasně znemožňuje pohyb hráče.

### 2.3.3 Grafika

Grafika postavy byla navržena, stejně jako všechny ostatní grafické části tohoto projektu, ve webové aplikaci Pixilart. Postava je vytvořena ve stylu pixel art, aby zapadala do celkového vizuálního stylu hry. Jednotlivé sprity jsou připraveny jako obrázky, které se následně načítají do programu a vykreslují na obrazovku podle aktuálního stavu postavy.

Pro většinu pohybů bylo vytvořeno více snímků, které určují, jak postava vypadá při různých akcích, například při stání, běhu, letu, pádu nebo skluzu. Naopak některé speciální stavy, jako je držení na stěně nebo nabíjení skoku, jsou reprezentovány pouze jedním obrázkem. Tento přístup zjednodušuje tvorbu grafiky a zároveň je pro hráče dostatečně přehledný.

### 2.3.4 Animace

Animace postavy jsou v této hře řešeny pomocí tzv. sprite sheetů, což jsou obrázky obsahující více snímků animace vedle sebe. Z těchto obrázků jsou jednotlivé snímky vyřezávány a ukládány do seznamů - (frames, runframes, flyframes, fallframes). Každý snímek je následně upraven na požadovanou velikost pomocí funkce pygame.transform.scale.

Snímky animace se mění pomocí proměnných current_frame, animation_timer a animation_speed. V každém snímku herní smyčky se zvyšuje hodnota animation_timer, a pokud překročí hodnotu 1, přesune se na další snímek:

Tímto způsobem vzniká plynulá animace, která se cyklicky opakuje.

Výběr aktuální animace spočívá na aktuálním stavu postavy. Program postupně kontroluje různé podmínky, například zda je postava ve stavu stun, nebo jestli se zrovna drží stěny (wall_grab), jestli zrovna letí nahoru ve vzduchu, padá nebo se pohybuje po zemi. Postupným procházením těchto podmínek se vybere odpovídající sprite:

Směř kterým se postava dívá je určen pomocí proměnné facing_right, která může být buď True nebo False. Pokud je False tak se nic neděje, protože ve snímcích je postava otočená do leva. Pokud je ale proměnná True znamená to že se snímek má horizontálně převrátit. Tato proměnná se mění např.: na základě rychlosti postavy, nebo z jaké strany se dotkla stěny

Pozice vykreslení spritu není na úplně stejném místě jako je hitbox, proto jsou použity proměnné sprite_offset_x a sprite_offset_y. Tyto hodnoty zajištují, že obrázkem je vždy na správné pozici relativně s hitboxem postavy.

## 2.4 Menu

Hlavní menu se skládá z pozadí vytvořeného v Pixilartu a několik tlačítek: „PLAY“, „SETTINGS“, „CONTROLS“ a „EXIT“. Na tlačítkách byl použit stažený font „Electroharmonix.otf“, který má Pseudo-Japonský styl písmen, takže hlavnímu menu přidává lepší vzhled a ladí s tématikou hry. Při pohybu myši se kontroluje její pozice a pomocí metody collidepoint se zjistí, zda se nachází na tlačítku.

Tato kontrola se používá hlavně pro zvýraznění tlačítek. Pokud hráč najede myší na tlačítko, tlačítko se zvýrazní, Díky tomu jsou tlačítka responsivnější a vypadají lépe.

Kliknutí na tlačítka je řešeno pomocí pygame.event.get(), kde se kontroluje událost MOUSEBUTTONDOWN. Pokud hráč klikne jakékoliv tlačítko myši, když se kurzor nachází na tlačítku, změní se hodnota proměnné game_state. Tato proměnná řídí, co se zrovna ve hře zobrazuje, například: přechod do hry, nastavení nebo ukončení hry.

Tento systém se nepoužívá jen v menu, ale i během hry. Hráč může kdykoliv zmáčknout klávesu Escape, čímž se game_state nastaví zpět na „MENU“ a hráč se vrátí do hlavního menu.

V nastavení má hráč možnost zvýšení nebo snížení hlasitosti zvuku pomocí slideru. Hráč může myší posunout jeho pozici a podle toho se mění hodnota sound_volume. Tato hodnota se pak využívá pro hlasitost přehrávání zvuků ve hře.

Další možnost v nastavení je zapnutí nebo vypnutí fullscreen režimu. To se řeší pomocí funkce pygame.display.set_mode, která změní velikost okna. Po přepnutí fullscreen režimu se také znovu přepočítají rozměry zobrazení, aby hře zůstaly správné proporce.

### 2.4.1 Font

Jak už je zmíněno, použitý font se jmenuje *Electroharmonix.* Tento font je dostupný pod licencí CC0, to znamená, že může být volně používán pro soukromé projekty, tak i pro veřejné. Díky tomu je perfektní pro ročníkovou práci.

## 2.5 Zvuky

Zvukové efekty ve hře byly vytvořeny webovou aplikací Bfxr, která je určená pro jednoduché generování a vytváření zvuků do 8bit pixel art her. Díky tomuto nástroji byly vytvořeny základní zvuky, konkrétně sound effect skoku a dopadu. Tyto zvuky jsou použity při hlavních herních akcích, díky tomu je hráč více vtáhnutý do hry.

Původně bylo plánováno použití FL Studia[10], program, který nabízí mnohem více možností pro tvorbu zvuků. Ten se ale ukázal jako zbytečně komplikovaný pro tento typ projektu a jeho rozsah. Z tohoto důvodu bylo rozhodnuto pro jednodušší řešení v s Bfxr, který umožňuje rychleji a efektivněji vytvořit jednoduché zvuky.

Vytvořené zvuky byly exportovány ve formátu wav a přidány do hry, kde se přehrávají při skoku nebo dopadu postavy.

Ve hře je také použitý sound effect známý jako vineboom, který se spustí, když se hráč omráčí.

[10] *IMAGE-LINE SOFTWARE. FL Studio. [online]. 2026. Dostupné z: https://en.wikipedia.org/wiki/FL_Studio. [cit. 2026-04-28].*

## 2.6 Problémy s vývojem

### 2.6.1 Problém s mechanikou wallgrab

S touto mechanikou byly ze začátku problémy s platformami, rozpoznání strany platformy, na kterou se hráč chtěl uchytit, rychlostí postavy a dalších problémů, musela být předělaná část kódu a lehce změnit strukturu kódu, konkrétněji v kontextu s horizontálními kolizemi. Kvůli obtížnosti byl využit na tuto funkci GitHub Copilot, který použil většinu již zadaného kódu a upravil kód, tak aby fungoval.

### 2.6.2 Debug

Z důvodů některých neefektivních částí kódu byla potřeba pro několik typů debuggů. Kvůli způsobu, na kterém jsou zavedeny platformy a celkově mapa, bylo pro zjednodušení pomocí umělé inteligence vyrobeny čtyři debug funkce, které výrazně pomohli celkovému vývoji – DEBUG_SHOW_PLATFORM_NUMBERS, DEBUG_SHOW_PLATFORM_COLLIDERS, DEBUG_HOT_RELOAD a DEBUG_TELEPORT_ON_CLICK. V kódu se již hot reload nenachází, jelikož kvůli němu nefungoval .exe soubor vyrobený pomocí Pyinstalleru. Zároveň se v kódu nachází ještě DEBUG_SHOW_HITBOX, který byl původně vytvořen bez umělé inteligence, ale poté byl přeformován.

### 2.6.3 Bugy

Ve hře se nachází bug, který hráč nemusí na první pohled zaregistrovat. Při pádu z platformy by si postava měla zachovat svou horizontální rychlost. Pokud ale hráč stojí na kraji platformy, pohybuje se směrem k jejímu okraji a zároveň začne držet mezerník, může dojít k situaci, kdy postava spadne, a její horizontální rychlost se nastaví na nulu.

Správně by se ale měla stát jedna ze dvou věci – buď se postava stihne zastavit a zůstane na platformě, nebo z ní spadne a zachová si momentum, které ale všechno kvůli bugu ztratí.

Další bug, který se ve hře objevil, nastal u jedné z překážek. Ta šla bez problémů překonat na počítači, na kterém probíhal vývoj, ale na školním počítači to možné nebylo. Překážka spočívala v tom, že hráč musel skočit a nechat se odrazit od stěny, aby dopadl na malou platformu, na kterou se nedalo dostat jiným způsobem.

Řešení tohoto problému spočívalo v tom, že byla platforma posunuta níže. Díky tomu bylo možné překážku překonat na všech počítačích, včetně školních nebo počítačů přátel, kteří si hru chtěli vyzkoušet.

# 3 Možná budoucí rozšíření

## 3.1 Příběh, konec a prodloužení mapy

Tato hra ještě nemá příběh, který byl plánovaný, jak je trochu naznačeno japonským hradem na konci mapy.

Hra nemá hotový konec, hráč může doskákat do konce, ale tam se nic nestane.

Mapa je relativně krátká a dala by se prodloužit za pomocí nových prvků mapy

## 3.2 Přidání nepřátel

Jelikož herní postava má katanu, bylo by možné přidat nepřátele a mechaniku, pomocí které by postava udělala útok a zároveň se posunula dopředu, což by, na malých platformách, vyžadovalo přesné vypočítání vzdálenosti, aby postava nespadla.

Kvůli tomu, že tento styl hry nemá životy, by jako trest byl nejspíše knock-back, kvůli kterému by postava mohla spadnout.

## 3.3 Přidání zkosených platforem a jiných elementů

Do hry je možné přidat zkosené platformy, které by mohli přidat na obtížnosti a komplexitě mapy. Je také možnost přidat např.: vítr v některých částech mapy, který by mohl s hráčem hýbat ze strany na stranu.

# 4 Závěr

Během vývoje se naskytly různé problémy, ať už jednoduché jako kolize, pohyb postavy a všelijaké mechaniky. Tyto problémy vedly k lepšímu pochopení, jak fungují lehké malé herní projekty a jak pracovat s knihovnou Pygame.

Cílem této ročníkové práce bylo vytvořit vlastní 2D plošinovou hru inspirovanou hrou Jump King, Tento cíl byl splněn a výsledkem je funkční hra obsahující jak základní mechaniky pohybu, tak i vlastní prvky, jako je wall jump a slide. Některé části hry nejsou úplně hotové, jako například příběh ale ty se můžou přidat v budoucnu.

# 5 Přílohy

## 5.1 Zdroje

### 5.1.1 Font

*1001 Fonts. Electroharmonix Font [online]. Dostupné z: https://www.1001fonts.com/electroharmonix-font.html [cit. 2026-04-28].*

### 5.1.2 Sound efekty

*BUSINESS GOOSE. Vine boom sound effect. [online]. 2022. Dostupné z: https://www.youtube.com/watch?v=Oc7Cin_87H4. [cit. 2026-04-28].*